

Blog Plan

How we plan weekly blogs on AOSP, HAL, BSP and embedded engineering: the JSON file, uploading it on the Blogs page, and how it is stored in Supabase.

WHERE

Dashboard →
Blogs

STORED IN

`public.blog_plan`

MIGRATION

`002_blog_plan.sql`

WHO CAN UPLOAD

Admins

IN SHORT

1

Write the plan

A JSON file with one brief per blog and its publish date.

2

Upload and check

On the Blogs page. Problems are listed before anything is saved.

3

Scheduled

Saved in Supabase; the page shows which blog goes out on which day.

CONTENTS

1 What the blog plan is

2 The JSON file

3 Uploading a plan

4 What the Blogs page shows

5 How it is stored in Supabase

6 Database reference

7 Table schemas

8 Uploading again

9 What is not automatic yet

10 Testing it

1 What the blog plan is

The blog plan is the list of blogs we want to publish and when. Each week we write it as a JSON file and upload it on the dashboard's **Blogs** page. The dashboard checks the file, saves it in Supabase and shows which blog goes out on which day.

Each entry is a brief, not the blog. It says what the blog is about and how to write it: topic, keywords, tone, length, target Android versions and reference links. The blog itself is written from the brief later (see section 9).

2 The JSON file

One file holds a list of blogs under `"blogs"`. Up to 60 blogs per file, up to 1 MB.

```
{
  "blogs": [
    {
      "id": "BLG-001",
      "topic": "AIDL vs HIDL: Writing a modern HAL in AOSP",
      "category": "HAL",
      "keywords": ["AIDL", "HIDL", "Treble"],
      "publish_date": "2026-10-12",
      "tone": "technical, beginner-friendly",
      "length": "1000-1500 words",
      "target_versions": ["Android 15"],
      "reference_urls": ["https://source.android.com/docs/core/architecture/aidl"]
    }
  ]
}
```

FIELD	REQUIRED	RULES
<code>id</code>	Required	Unique in the file. Up to 40 letters, digits, <code>-</code> or <code>_</code> . Uploading the same id again updates that blog.
<code>topic</code>	Required	10 to 200 characters.
<code>category</code>	Required	Usually AOSP, HAL, BSP, Embedded, Kernel, Drivers, Security or Tooling. Others are allowed with a warning.
<code>keywords</code>	Required	1 to 10 words or phrases.
<code>publish_date</code>	Required	<code>YYYY-MM-DD</code> , today or later.
<code>tone</code>	Optional	Text, up to 100 characters.
<code>length</code>	Optional	Text, up to 100 characters, e.g. <code>"1000-1500 words"</code> .
<code>target_versions</code>	Optional	List of texts, e.g. <code>["Android 15", "Android 16"]</code> .
<code>reference_urls</code>	Optional	Up to 10 links, each starting with <code>http://</code> or <code>https://</code> .

3 Uploading a plan

Only admins see the upload button. The backend checks the admin role in the database on every upload, so the button being visible is not what allows it.

- 1 Open **Blogs** and click **Upload blog plan** at the top of the Blog Plan card.
- 2 Choose the `.json` file. The dashboard checks it first and **saves nothing yet**.
- 3 A preview lists every blog with its date and what will happen to it.
- 4 Fix any problems in the file and upload it again; **Import** stays disabled while there are problems. Warnings (for example two blogs on the same day) don't stop the import.
- 5 Click **Import**. The page shows *"Imported: 4 new, 1 updated."* and the schedule refreshes.

THE PREVIEW

DATE	ID	TOPIC	CATEGORY	WILL BE
Mon, Oct 12	BLG-001	AIDL vs HIDL: Writing a modern HAL in AOSP	HAL	New
Thu, Oct 15	BLG-002	Bringing up a new board: a BSP checklist	BSP	Update
Mon, Oct 19	BLG-003	What changes for vendors in Android 16 GKI	Kernel	Skipped (posted)

What the preview means

- **New** not in the plan yet.
- **Update** a blog with this id is already in the plan; its details are replaced.
- **Skipped (posted)** already posted, so it is never changed.

How problems look

BLG-102 • publish_date: Use a real date written as YYYY-MM-DD.

BLG-104 • reference_urls: Give at most 10 links, each starting with `http://` or `https://`.

4 What the Blogs page shows

BLOG PLAN CARD (AFTER UPLOADING THE SAMPLE FILE)

Planned blogs

6

This week

0

Posted

0

Next blog

Mon, Oct 12
AIDL vs HIDL...

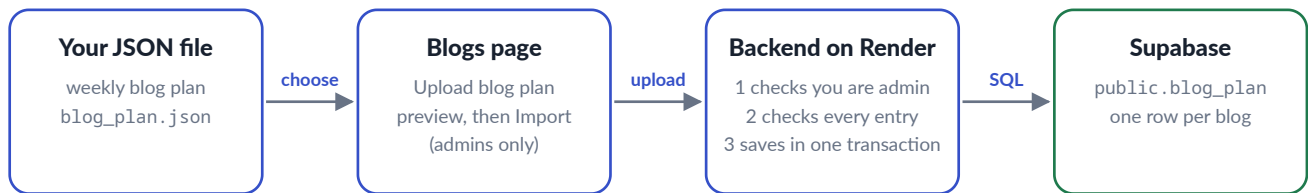
DATE	BLOG	CATEGORY	STATUS
WEEK OF OCT 12			
Mon, Oct 12	AIDL vs HIDL: Writing a modern HAL in AOSP	HAL	Planned
Thu, Oct 15	Bringing up a new board: a practical BSP checklist	BSP	Planned
WEEK OF OCT 19			
Mon, Oct 19	What changes for vendors in Android 16 GKI	Kernel	Planned

STATUS	MEANING
Planned	In the plan, not written yet.
Written	A blog post has been written from it, not posted yet.
Posted	Its blog is published.
Missed	The date has passed and it isn't posted.
Cancelled	Taken off the schedule. Uploading it again brings it back.

This week counts Monday to Sunday. Click a row to see the full brief (keywords, tone, length, target versions, reference links). Admins can **Cancel** a blog that isn't posted yet.

5 How it is stored in Supabase

Each blog in the JSON becomes **one row** in the Supabase table `public.blog_plan`.



The browser never writes to Supabase directly; only the backend does, through its database connection. Row level security is on with no policies, so the website's public Supabase key can't read or change the table.

What one row looks like

The `BLG-001` entry from the sample file is saved as:

COLUMN	VALUE
<code>id</code>	<code>3f9c...</code> generated by the database; used in the dashboard's links
<code>external_id</code>	<code>BLG-001</code> , the JSON id (unique)
<code>topic</code>	AIDL vs HIDL: Writing a modern HAL in AOSP
<code>category</code>	HAL
<code>keywords</code>	<code>{AIDL,HIDL,Treble}</code> (a list)
<code>publish_date</code>	2026-10-12
<code>tone</code> / <code>length</code>	technical, beginner-friendly · 1000-1500 words
<code>target_versions</code>	<code>{Android 15}</code>
<code>reference_urls</code>	<code>{https://source.android.com/docs/core/architecture/aidl}</code>
<code>status</code>	<code>planned</code> (or <code>written</code> , <code>posted</code> , <code>cancelled</code>). Missed is worked out from the date, never stored.
<code>post_id</code>	Empty until a blog post is written from it (<code>content_posts.id</code>)
<code>uploaded_by</code>	Email of the admin who last uploaded it
<code>created_at</code> / <code>updated_at</code>	When it was first added and last changed

What changes a row

- **Same id uploaded again:** the row is updated, never duplicated.
- **Posted blog:** never changed by an upload.
- **Cancel:** sets `status` to `cancelled`. Rows are never deleted.

Created automatically

The backend runs `create table if not exists` the first time anyone opens the Blogs page or uploads. Running `002_blog_plan.sql` yourself is optional; doing both is harmless.

Looking at it in Supabase

Table Editor → `blog_plan`, or in the SQL Editor:

```
select external_id, publish_date, topic, category, status
from blog_plan
order by publish_date;
```

6 Database reference

“This repo” is `~/Marketing_Agent/dashboardbackend`; “agent repo” is `~/Marketing-AI-Agent`.

Migrations

FILE	CREATES	NEEDED FOR	RUN BY HAND?
<code>002_blog_plan.sql</code> this repo · migrations/	<code>blog_plan</code> , its index, row level security	The blog plan	Optional: the backend runs the same SQL on first use
<code>001_dashboard_presence.sql</code> this repo · migrations/	<code>dashboard_presence</code>	One sign-in per account, checked on every request	Optional: the backend runs it on first use
<code>0002_core.sql</code> and later agent repo · supabase/migrations/	The agent's tables, including <code>content_posts</code>	Blog posts the plan links to	Already applied; not changed by the blog plan
Supabase Auth built in	<code>auth.users</code> , <code>auth.sessions</code>	Sign-in and roles	Nothing to run

`002_blog_plan.sql` needs `content_posts` to exist first, because `blog_plan.post_id` points to it. It already exists in this project (the agent created it).

Tables the blog plan uses

TABLE	OWNED BY	ACCESS	WHY
<code>public.blog_plan</code>	This repo	<code>Read + write</code>	The plan itself, one row per blog

TABLE	OWNED BY	ACCESS	WHY
<code>public.content_posts</code>	Agent repo	Read	Shows Written / Posted and links to the posted blog (through <code>post_id</code>)
<code>auth.users</code>	Supabase Auth	Read	Your role (only admins upload or cancel) and whether your account was removed
<code>auth.sessions</code>	Supabase Auth	Read	Whether your sign-in session still exists
<code>public.dashboard_presence</code>	This repo	Read + write	One sign-in per account (every request)

What each action does

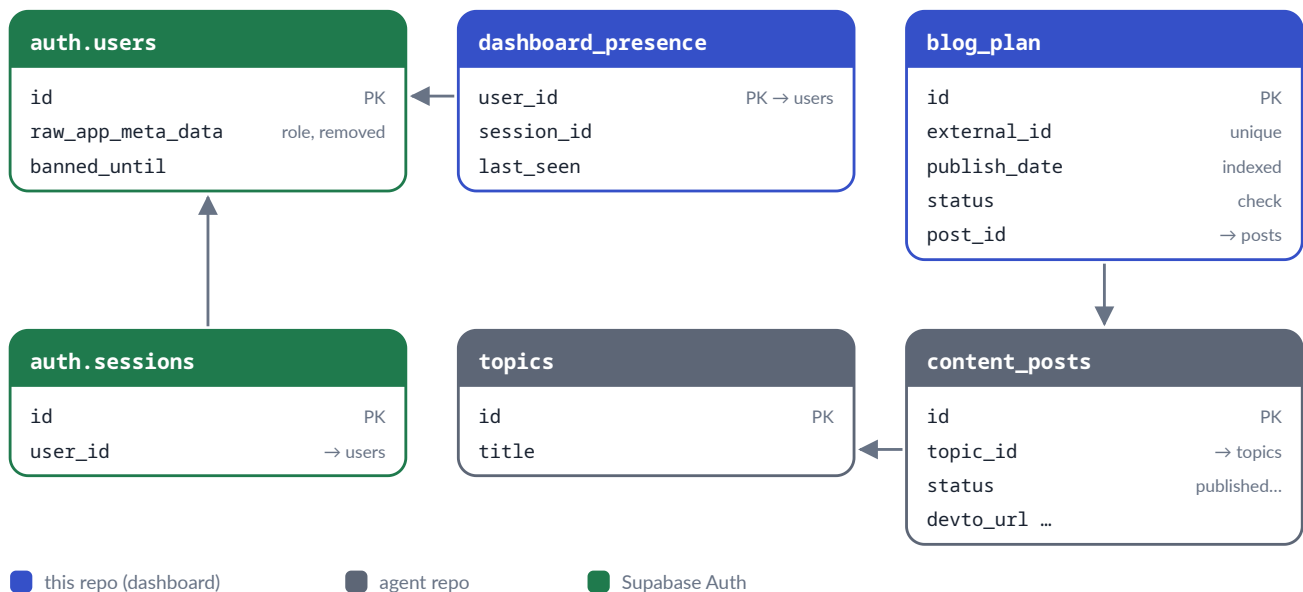
ON THE BLOGS PAGE	BACKEND ROUTE	DATABASE
Open the page	GET <code>/blogs/plan</code>	Reads <code>blog_plan</code> joined with <code>content_posts</code>
Choose a file (the check)	POST <code>/blogs/plan/import</code> <code>?dry_run=true</code>	Reads which ids already exist; writes nothing
Import	POST <code>/blogs/plan/import</code>	One transaction: locks the existing rows, then insert or update each blog by <code>external_id</code> (posted ones skipped)
Click a row	GET <code>/blogs/plan/{id}</code>	Reads one row
Cancel	POST <code>/blogs/plan/{id}/cancel</code>	Sets <code>status = 'cancelled'</code>

Every route also checks your sign-in first (`auth.users` , `auth.sessions` , `dashboard_presence`). Values from the file are always sent as query parameters, never pasted into the SQL.

Rules the database enforces

- `external_id` is **unique**: the same JSON id can only exist once.
- `status` must be `planned` , `written` , `posted` or `cancelled` .
- `post_id` must point to a real `content_posts` row; if that post is deleted, `post_id` becomes empty.
- `publish_date` is **indexed**, so sorting the schedule stays fast.
- **Row level security** is on with no policies: only the backend's database connection can use the table.

7 Table schemas



How the tables connect. Arrows point from the column that refers to another table.

public.blog_plan this repo · migrations/002_blog_plan.sql (the whole file)

```
-- Blog plan (app/blog_plan.py, BLOG_PLAN.md): one row per blog brief uploaded as JSON on the Blogs
page.
-- The backend also creates this on first use; running it here by hand is optional. Run once in the
Supabase SQL editor.
create table if not exists public.blog_plan (
  id                uuid primary key default gen_random_uuid(),
  external_id       text not null unique,           -- the JSON "id", e.g. BLG-001
  topic             text not null,
  category          text not null,
  keywords          text[] not null default '{}',
  publish_date      date not null,
  tone              text,
  length            text,
  target_versions   text[] not null default '{}',
  reference_urls    text[] not null default '{}',
  status            text not null default 'planned'
                  check (status in ('planned', 'written', 'posted', 'cancelled')),
  post_id           uuid references public.content_posts (id) on delete set null, -- the blog written
  from it
  uploaded_by       text,                          -- email of the admin who uploaded it
  created_at        timestampz not null default now(),
  updated_at        timestampz not null default now()
);
create index if not exists blog_plan_publish_date_idx on public.blog_plan (publish_date);

-- Only the dashboard backend (database connection) uses this table. RLS with no policies keeps it out
of reach of
-- the browser's anon / authenticated keys.
alter table public.blog_plan enable row level security;
```

public.dashboard_presence this repo · migrations/001_dashboard_presence.sql (the whole file)

```
-- One sign-in per account (app/auth.py): which session is using each account, and when it was last
seen.
-- Run once in the Supabase SQL editor.
create table if not exists public.dashboard_presence (
  user_id      uuid primary key references auth.users (id) on delete cascade,
  session_id   uuid not null,
  last_seen    timestampz not null default now()
);

-- Only the dashboard backend (database connection) uses this table. RLS with no policies keeps it out
of reach of
-- the browser's anon / authenticated keys.
alter table public.dashboard_presence enable row level security;
```

public.content_posts agent repo · 0002_core.sql + 0003_daily_agent.sql, as it is now

The blog plan only reads **id**, **status** and the three URL columns. A post's **published** status shows as **Posted** ; any other status of a linked post shows as **Written** .

```
create table content_posts (
  id                uuid primary key default gen_random_uuid(),
  topic_id          uuid references topics (id),
  title             text,
  body_md           text,
  status            text not null default 'idea'
                  check (status in ('idea', 'drafted', 'in_review', 'approved',
                                     'published', 'rejected', 'publish_failed')),
  reviewer_note     text,
  cms_url           text,
  devto_url         text,
  linkedin_url      text,
  idempotency_key   text unique,          -- 'blog-YYYY-MM-DD' for the agent's daily blog
  metrics           jsonb,
  embedding          vector(1024),
  embedding_model    text,
  metadata          jsonb,               -- added in 0003_daily_agent.sql
  tags              text[],              -- added in 0003_daily_agent.sql
  created_at        timestampz not null default now(),
  updated_at        timestampz not null default now(), -- set by trigger touch_updated_at
  published_at      timestampz
);
create index content_posts_status_idx on content_posts (status);
-- Row level security on; signed-in team members may read (policy content_posts_team_read).
```

auth.users and **auth.sessions** Supabase Auth · managed by Supabase

No migration of ours changes these. The columns the backend reads:

TABLE	COLUMN	USED FOR
auth.users	id	The signed-in person (the token's sub)
auth.users	raw_app_meta_data	role = admin allows upload and cancel; removed blocks a removed account
auth.users	banned_until	A banned account is refused
auth.sessions	id, user_id	Whether the token's sign-in session still exists

8 Uploading again

- A blog with the same `id` is **updated**: topic, date, keywords and the rest are replaced.
- A blog whose post is already **published** is never changed and shows as Skipped.
- A **cancelled** blog uploaded again goes back to Planned.
- Blogs in the database but not in the new file are left as they are. To remove one, cancel it.

9 What is not automatic yet

Nothing writes a blog from a brief or publishes it on its date yet. `post_id` and `status` are there so that can be added. The options:

OPTION 1

The agent writes it

The agent's daily blog (`agent/blog.py`) picks its own topic today. It could take the next brief due within a few days, write the post into `content_posts`, and set `post_id` and `status = 'written'`.

OPTION 2

A job on Render

A scheduled job publishes posts whose brief's date has come, then sets `status = 'posted'`.

OPTION 3

By hand

Write and post the blog yourself, then set `post_id` so the dashboard shows it as posted.

With option 1, decide whether the agent should stop picking its own topics, or both will produce blogs.

10 Testing it

FILE (IN DASHBOARDBACKEND/SAMPLES/)	WHAT IT DOES
<code>blog_plan.sample.json</code>	6 valid blogs, Mondays and Thursdays from Oct 12 to Oct 29, 2026. Upload it: the preview shows 6 New , and after Import the card shows 6 planned blogs. Upload it again: 6 Update .
<code>blog_plan.invalid.json</code>	One mistake per blog: missing topic, impossible date, past date, <code>ftp://</code> link, duplicate id, empty keywords, unusual category. Every problem is listed and Import stays disabled.

The sample dates are fixed. After Oct 29, 2026 they are in the past and the valid file will be refused; change the dates to test again.

Automated tests: `pytest tests/test_blog_plan.py` covers validation with both sample files, import, dry run, admin-only access, cancel, the statuses and the summary counts.